

Actuate

Technical Implementation Specification

*Layer-by-Layer Library, Model & Framework Choices for the
CLI/API-First Multimodal Data Pipeline*

This document specifies the concrete technology stack for building Actuate as a CLI/API-first pipeline. For every layer it names the exact libraries and models to use, states why each is the right choice as of 2026 (with the runner-up and why it lost), and flags where the current codebase already uses something that should be upgraded. It assumes the architecture from the v1 and v2 design documents and does not re-argue it here.

0 Stack Principles (read first)

Five rules govern every choice in this document, so they aren't re-justified per layer:

- **CLI/API-first, library-shaped core.** The pipeline is a Python package invocable as a CLI and importable as a library. No web framework in the core. A thin FastAPI layer (already built) wraps it for remote/job use; the browser dashboard is a separate consumer, never a dependency.
- **Foundation models over bespoke training.** Perception models are swappable, off-the-shelf, open-weights checkpoints. Actuate's value is orchestration, fusion, calibration, retargeting, and certification — not re-deriving hand pose estimation. Every model slot is an interface with a default, not a hard dependency.
- **Metric and uncertainty-aware wherever possible.** Prefer models that output real metric scale and a confidence/uncertainty signal, because both feed the certification layer directly. A model that only gives relative output forces a downstream calibration step and loses trust signal.
- **Pin versions, isolate heavy CUDA deps.** Perception model repos are mutually version-sensitive; each runs behind a stable interface so a version conflict in one never breaks the pipeline contract.
- **Verify against real data, never assume.** Consistent with the whole build history: a library is only 'chosen' once it's run on a real session, not once it's added to requirements.

1 Core Language, Packaging & CLI Layer

1.1 Language and package management

Concern	Choice	Why this, not the alternative
Language	Python 3.11+	Non-negotiable — every perception/retargeting model, MuJoCo, Isaac, LeRobot, and the ML ecosystem are Python-native. 3.11 for speed + stable typing; not 3.12+ yet since some CUDA-bound wheels lag.
Env / deps	uv + pinned requirements per component group	uv resolves and installs far faster than pip and handles the conflicting CUDA-dep groups cleanly; not Poetry (slower, heavier lockfile churn for this many optional heavy groups).
Packaging	src-layout Python package + pyproject	Importable-as-library AND installable-as-CLI from one source tree; not a loose script collection (the v1 pipeline's original shape), which doesn't compose into an SDK.

1.2 CLI framework

Choice	Why
Typer (over argparse / Click)	Typer gives typed, self-documenting sub-commands with near-zero boilerplate and auto-generated help/completion — ideal for an <code>actuate process / retarget / export</code> command tree. It's built on Click so nothing is lost; argparse would be far more verbose for a multi-command tool.

```
actuate process ./raw/session_007 --rig umi_gripper --depth-mode monocular
actuate retarget session_007 --embodiment franka_dual --sim-validate
actuate export session_007 --format lerobot_v3 --out ./delivery
```

1.3 Config, jobs, storage

Concern	Choice	Why
Config	Pydantic Settings + per-session config file	Typed, validated config with env-var override; replaces the v1 global <code>config.py</code> constants with something the CLI and API share safely.
Job queue	Celery + Redis (or RQ for smaller scale)	Perception/retarget jobs are long (minutes–hours) and must not block; a proven broker-backed queue with retries and a persistent result backend, matching the resumable-status requirement. Not in-memory-only (the current service's known fragile point).
Data store	Postgres (metadata) + S3/GCS (artifacts)	Structured session/episode/run records in Postgres; heavy artifacts (video, point clouds, depth) in object storage. Standard, boring, correct — no exotic DB needed.

2 Layer 0 — Ingestion & Sync: Stack

Slot	Choice	Why this is best here
Container	MCAP (foxglove/mcap)	Purpose-built for multi-topic, timestamped robotics logs; the emerging raw-provenance standard and what UMI-style specs mandate. Not raw MP4+CSV (v1's shape) — that can't hold N heterogeneous timestamped channels cleanly.
Video I/O	PyAV (ffmpeg bindings)	Direct, frame-accurate access to timestamps/PTS and codecs without shelling out to ffmpeg text parsing; more robust than ffmpeg-python for precise per-frame timing which sync depends on.
Numerics/tables	NumPy + Polars	Polars over pandas for the large per-frame tabular joins (sync, resampling) — far faster and lower-memory on the million-row scale a session hits; pandas kept only where an external lib needs it.
Array store	HDF5 (h5py) / Zarr	Dense per-frame arrays (synced IMU, poses) in HDF5 for single-session locality; Zarr when chunked cloud-native access is needed for big batches.
Sync method	Hardware-timestamp alignment + interpolation on Polars	Use real hardware timestamps where present; interpolate against highest-rate stream otherwise. This is pipeline logic, not a library — the value is the rig-aware adapter, not a dependency.

Codebase note

v1 ingestion reads raw.mp4 + imu.json. This is a real rework toward MCAP + the canonical multi-stream schema (Architecture v2 §6). The existing PyAV-style timestamp handling and Polars-friendly sync logic port forward; the single-video/single-IMU assumption does not.

3 Layer 1 — Perception & Metric Reconstruction: Stack

This is the most model-heavy layer. Every slot below is an off-the-shelf, open-weights foundation model behind a stable interface. Choices reflect the 2026 state of the art, verified against current benchmarks.

3.1 Hand pose & mesh

Choice	Why it wins	Runner-up / why not
WiLoR	Best current in-the-wild 3D hand reconstruction: outperforms HaMeR on FreiHAND/HO-3D (PA-MPJPE 5.5 vs 6.0, higher F-scores), has built-in real-time hand localization (no separate detector), and ships robust open-source weights + MANO output.	HaMeR — strong and widely cited, but needs a separate detector and trails WiLoR on the standard benchmarks. Kept as a fallback interface.

3.2 Metric depth & camera intrinsics

Choice	Why it wins	Runner-up / why not
UniDepthV2	Predicts <i>metric</i> 3D directly (not relative), <i>estimates camera intrinsics from the image itself</i> — which is exactly the approximated-calibration fallback path — and outputs a per-pixel <i>uncertainty</i> level that feeds certification confidence directly. Zero-shot SOTA across 10 depth datasets.	Depth-Anything-V2-Metric (current pipeline default) — good, but the base model is relative; the metric variant is domain-tuned (indoor/outdoor split) and gives no intrinsics estimation or uncertainty. This is a concrete upgrade.

3.3 Object detection, segmentation & tracking

Slot	Choice	Why
Open-vocab detect	Grounding DINO	Text-prompted detection so operator task metadata or a task description drives what's found; the field standard, already in the pipeline.
Segment + track	SAM 2	Promptable video segmentation with mask propagation across frames — one encode seeds all objects per chunk; already integrated, keep.
6-DoF object pose	FoundationPose (where mesh available)	Strong zero-shot 6-DoF pose/tracking for known/reconstructed objects; used where object geometry matters for contact reasoning.

3.4 Body pose & gravity (when needed)

Slot	Choice	Why
Full-body (context)	SMPL-X via a current regressor	Only when body context matters (some head-mounted third-person capture); most manipulation cases need hands + objects only.
Camera/gravity	UniDepthV2 intrinsics + gravity from static-frame estimation	Reuses the depth model's intrinsics output; gravity needed for retargeting root-frame + ego reframing.

Codebase note

Current perception uses MediaPipe Hands (2D-ish landmarks) and Depth-Anything-V2-Metric. Moving hand pose to WiLoR (true 3D MANO mesh) and depth to UniDepthV2 (metric + intrinsics + uncertainty) are the two highest-value model swaps — both feed retargeting and certification quality directly. MediaPipe can stay as a fast pre-filter in the two-pass activity scan.

4 Layer 2 — Sensor Fusion: Stack

This layer is mostly pipeline logic, not third-party models — its value is the trust-weighted arbiter. Libraries are lightweight.

Slot	Choice	Why
State machine	Plain typed Python (no framework)	The interaction-state machine (static/grasped/moving + dominant hand) is small, deterministic, and testable as pure functions — a workflow library would add nothing but weight.
Signal processing	SciPy + NumPy	Schmitt-trigger gating, smoothing, morphological close-and-drop on primitive streams — all standard signal ops.
Sensor decode	Rig-specific parsers (glove flex, gripper aperture)	Small adapters per hardware type; the trust hierarchy (hardware > vision) lives here as logic, not a dependency.

Codebase note

The v1 primitive/confidence code is the foundation. The rework (Architecture v2 §5) is exposing a clean per-finger contact-confidence signal — extension of existing SciPy/NumPy logic, no new heavy dependency.

5 Layer 3 — Canonical Representation: Stack

Slot	Choice	Why
Schema definition	Pydantic models + a frozen, versioned JSON schema	Typed, validated, self-documenting canonical episode schema shared by every downstream consumer; version-pinned so exporters build against a stable contract (Architecture v2 §6).
Geometry types	NumPy + a small SE(3) util (or roma / pytorch3d transforms)	Consistent right-handed, gravity-aligned SE(3) handling; roma is a light, correct rotation library avoiding hand-rolled quaternion bugs.
Serialization	Parquet (per-frame tabular) + HDF5/npz (dense arrays)	Columnar Parquet is the on-disk form LeRobot v3 itself uses, so the canonical store is already close to the delivery form — minimal transcription later.

6 Layer 4 — Certification: Stack

Slot	Choice	Why
Scoring logic	Pure Python + NumPy	The certificate is a weighted composite of measured quantities (sync drift, calibration completeness, perception + fusion confidence, IK feasibility) — deterministic math, fully testable, no ML.
Face redaction	RetinaFace or MediaPipe Face Detection	Bystander-face blurring for the PII gate; MediaPipe variant avoids pulling a second DL framework if TF-heavy RetinaFace is a burden.

Slot	Choice	Why
Text/badge redaction	EasyOCR	Detects visible text/badges for redaction; already integrated and verified against real footage.
Gate enforcement	Pipeline logic (fail-closed)	Consent + quality gates are hard, code-level blocks — the most important non-ML component in the whole system.

7 Layer 5 — Retargeting Engine: Stack

The largest net-new workstream. Split by branch (Architecture v2 S7).

7.1 Arm/wrist branch

Slot	Choice	Why
Root-frame estimator	SE(3)-equivariant net (Vector Neuron layers), flow-matching, sim-trained	EgoInfinity-validated design; VN layers give exact rotation-equivariance, flow-matching captures root-pose ambiguity under partial observation. Built in PyTorch.
IK solver	Pinocchio (+ optionally cuRobo for GPU batch IK)	Pinocchio is the standard fast rigid-body/IK library with analytical derivatives; cuRobo when high-throughput GPU-parallel IK over many candidates/frames is needed.

7.2 Finger/dexterous branch

Slot	Choice	Why
Neural finger retarget	GeoRT (public code)	Fast, principled neural human-to-robot hand retargeting; the current practical SOTA with usable code for the finger map.
Functional objective	DexMachina-style (public code + sim benchmark)	Contact-guided, object-state-tracking objective for robustness across differing hand kinematics; released code and sim benchmark make it buildable.
Objective tuning	Informed by retargeting-objectives ablation (Xin et al.)	Empirical priors on which cost terms matter, so the finger objective is evidence-tuned.

7.3 Simulation validation

Slot	Choice	Why
Primary sim	MuJoCo 3.x	Fast, accurate contact dynamics, the de-facto manipulation-research simulator; ideal for feasibility replay (collision, joint limits, contact stability, no-slip). LeRobot/LIBERO already standardize on it.
Scale/photoreal sim	NVIDIA Isaac Sim / Isaac Lab	When GPU-parallel validation at scale or photorealistic rendering for embodiment onboarding is needed; heavier, used selectively.
Robot models	URDF/MJCF per embodiment	Standard kinematic/collision descriptions driving both IK and sim replay.

8 Layer 6 — Language & Task Annotation: Stack

Slot	Choice	Why
VLM annotator	Hosted VLM API (Claude / Gemini-class), frame-sampled	Already validated in the pipeline (real API calls, ~\$2.85/hr-of-video). Grounds instructions in actual sampled frames; open task-vocab identification + hallucination check. Hosted, so no local VLM GPU burden.
Frame sampling	Phase-boundary keyframe sampling (contact-sheet style)	Sending a few timestamped keyframes at phase boundaries is cheaper and more accurate than long frame sequences — the field finding this pipeline already uses.
Action ontology	Closed enum + Pydantic validation	Fixed atomic-action vocabulary beneath open-vocab task names; closed set is what makes cross-dataset comparison possible.

Self-hosted open-weights VLMs (Qwen-VL-class) are the alternative if API cost or data-residency demands it — same interface, trades API spend for GPU + ops. Keep the annotator behind an interface so this is a config switch, not a rewrite.

9 Layer 7 — Packaging & Delivery: Stack

Slot	Choice	Why
Primary export	LeRobot (lerobot library) → Dataset v3	Write the exact consolidated format VLA training expects (chunked Parquet + per-camera MP4 + metadata), using the reference library itself so schema correctness is inherited, not re-implemented.
Secondary export	RLDS (TFDS-based writer)	For Open-X-convention stacks (OpenVLA/Octo-family); episodes as timestep sequences.
Dense/raw	HDF5 + MCAP provenance passthrough	robomimic-style HDF5 where asked; original MCAP raw shipped as required provenance.
Dedup	CLIP / video-embedding + FAISS	Embed sampled frames, index with FAISS for fast near-duplicate detection across episodes before splits — prevents train/val leakage.
Splits/stats	Polars + NumPy	Stratified episode-level splits and per-dataset normalization statistics (which VLA training requires) computed and shipped in metadata.

Verification requirement

Per the project's standing discipline: the LeRobot v3 exporter must be verified by loading its output with LeRobot's own loader and a real training step, not merely asserted schema-correct. This is exactly the class of gap (mock vs. real) caught repeatedly during the build.

10 Cross-Cutting: Serving, Observability, Testing

Concern	Choice	Why
API wrapper	FastAPI (already built)	Thin async layer exposing run/status/results/consent/export over HTTP for remote + app use; core stays framework-free.
GPU serving	Containerized workers on serverless GPU (Modal / RunPod) or persistent cloud GPU	Perception + retargeting are GPU-bound, bursty per episode — serverless GPU keeps spend proportional to jobs; not a notebook (no stable endpoint, expires).
Observability	Structured logging + per-stage timing + run lineage in Postgres	Every stage writes a persistent run record (not log-scraping) — fixes the current service's fragile regex-parse-the-log status approach.
Experiment tracking	Weights & Biases (optional, for retargeting model training)	Standard for the sim-trained root-frame + finger nets; only relevant to the model-training sub-workstreams.
Testing	pytest + synthetic fixtures + one real-session smoke test per layer	The exact discipline that caught every real bug in this project: unit tests plus a real-data run before any layer is 'done'.

11 Consolidated Stack Table

Layer	Primary tech / models
Core / CLI	Python 3.11, uv, Typer, Pydantic, Celery+Redis, Postgres, S3/GCS
L0 Ingest/Sync	MCAP, PyAV, Polars, NumPy, HDF5/Zarr
L1 Perception	WiLoR (hand), UniDepthV2 (metric depth+intrinsic+uncertainty), Grounding DINO + SAM 2 (objects), FoundationPose (6-DoF), SMPL-X (opt.)
L2 Fusion	SciPy/NumPy, typed state machine, rig sensor parsers
L3 Canonical	Pydantic schema (versioned), roma SE(3), Parquet + HDF5
L4 Certification	NumPy scoring, RetinaFace/MediaPipe + EasyOCR (PII), fail-closed gates
L5 Retargeting	VN + flow-matching net (PyTorch), Pinocchio/cuRobo IK, GeoRT + DexMachina (dexterous), MuJoCo / Isaac Sim validation
L6 Language	Hosted VLM API, phase-boundary keyframes, closed action ontology
L7 Packaging	LeRobot v3, RLDS/TFDS, HDF5, CLIP+FAISS dedup, Polars splits/stats
Cross-cutting	FastAPI, serverless GPU workers, Postgres run lineage, pytest + real-session smoke tests

Every model choice here is an interface with a default, not a hard commitment — the field moves fast, and the swap-cost is deliberately low. Version pins, benchmark claims, and 'best' designations reflect early-to-mid 2026 and should be re-checked before each is locked in. Nothing is chosen until it has run on a real session.